XI'AN JIAOTONG-LIVERPOOL UNIVERSITY 西 交 利 物 浦 大 学

COURSEWORK SUBMISSION COVER SHEET

Name	Li(Surname)	Hanqing(Other Names)
Student Number	2363017	
Programme	Numerical Differentiation Beyond the Real Plane	
Module Title	Numerical Analysis	
Module Code	Mth208	
Assignment Title	Numerical Differentiation Beyond the Real Plane	
Submission Deadline	2026.5.31	
Module Leader	Niu Qiang	

By uploading or submitting this coursework submission cover sheet, I certify the following:

- I have read and understood the definitions of collusion, copying, plagiarism, dishonest use of data, and academic offences involving Artificial Intelligence (AI) as outlined in the Academic Integrity Policy of Xi'an Jiaotong-Liverpool University.
- This work is my own, original work produced specifically for this assignment. It does not misrepresent the work of another person or institution, nor does it present the work generated by AI as my own. Additionally, it is a submission that has not been previously published, or submitted to another module.
- This work is not the product of unauthorized collaboration between myself and others.
- This work is free of embellished or fabricated data.

I understand that collusion, copying, plagiarism, dishonest use of data, academic offences involving AI, and submitting procured work are serious academic misconduct. By uploading or submitting this cover sheet, I acknowledge that I am subject to disciplinary action if I am found to have committed such acts.

Signature ……Hanqing Li…… Date ……*2026.5.27*……

For Academic Office use:	Date Received	Working Days Late	Penalty[1]

[1] Please refer to clause 43 (i), (ii) and (iii) of the Code of Practice on Assessment for the standard system of penalties for the late submission of assessment work based on **working days**.

Numerical Differentiation Beyond the Real Plane

Numerical Analysis (MTH208)

May 27 2026

Abstract

This report examines the numerical stability of several derivative approximation methods, with a particular focus on the limitations of real axis finite difference formulas and the advantages of complex plane approaches. The forward finite difference method is first studied through Taylor expansion, which shows that its leading truncation error is $O(h)$, while its round off error increases when the step size becomes too small. The complex step derivative approximation is then derived from the Taylor expansion of $f(x + ih)$. This method has $O(h^2)$ truncation error and avoids the subtraction of two nearly equal real numbers. For $f(x) = \sin x$ at $x = \pi/4$, the forward finite difference method achieves a minimum absolute error of 5.287×10^{-10} at $h = 1.874 \times 10^{-9}$, while the complex step method is displayed as reaching zero error in MATLAB double precision output. For high order derivatives, Cauchy's integral formula is combined with the fast Fourier transform to compute derivatives from function values sampled on a circular contour. In the fifth derivative test with exact value $f^{(5)}(0) = -164$, the FFT based Cauchy method reaches an error of 1.1369×10^{-13}, whereas the fifth derivative finite difference stencil only achieves a best error of 3.309×10^{-2}. These results show that complex plane and FFT based methods provide substantially better stability than conventional finite difference schemes, especially for small step sizes and high order derivatives.

1 Introduction

Numerical differentiation is a basic but delicate problem in numerical analysis. In exact mathematics, a derivative is defined by a limiting process. In numerical computation, however, this limit must be replaced by a finite step size and evaluated using floating point arithmetic. This creates an unavoidable tension between truncation error and round off error. A large step size gives a poor local approximation to the derivative, while an excessively small step size can make the computation unreliable because nearly equal function values are subtracted. This difficulty becomes more serious for high order derivatives, where finite difference coefficients and stencil formulas may amplify numerical perturbations [1].

The forward finite difference method is one of the simplest derivative approximations, and it provides a useful starting point for studying this trade off. Its error can be understood directly from Taylor expansion: as the step size decreases, the truncation error

initially becomes smaller, but the subtraction $f(x+h) - f(x)$ eventually causes loss of significant digits. The complex step derivative approximation offers a more stable alternative for first derivatives. Instead of forming a real difference quotient, it evaluates the function at $x + ih$ and extracts derivative information from the imaginary part. This method avoids the direct subtraction of two nearly equal real numbers and is therefore much less vulnerable to subtractive cancellation [2].

For analytic functions, differentiation can also be studied through the complex plane. Cauchy's integral formula expresses derivatives in terms of values of the function on a closed contour surrounding the point of interest. When the contour is chosen as a circle and sampled at equally spaced points, the derivative formula becomes closely related to Fourier coefficients. This makes it possible to compute high order derivatives efficiently using the fast Fourier transform. Such complex contour and spectral ideas are central to numerical differentiation of analytic functions and to FFT based differentiation methods [3, 4].

The purpose of this report is to compare these ideas both theoretically and numerically. First, Taylor series expansions are used to derive the leading error terms of the forward finite difference and complex step methods. Then MATLAB experiments are carried out for $f(x) = \sin x$ at $x = \pi/4$ to compare their error behaviour over a wide range of step sizes. For high order differentiation, the fifth derivative of an analytic test function is computed using an FFT based Cauchy method and compared with a fifth derivative central finite difference stencil on the real axis. This comparison highlights why complex plane methods can achieve much greater numerical stability than conventional finite difference formulas, especially when high order derivatives or very small step sizes are involved.

2 Methodology and Algorithms

2.1 Task 1: Taylor Expansion and Error Mechanisms

This section derives the main error properties of the forward finite difference method and the complex step derivative approximation. The purpose is to show how truncation error arises from Taylor series expansion, and why the complex step method avoids the subtractive cancellation that affects real finite difference formulas. The complex step method used here follows the derivative approximation discussed by Martins et al. [2].

2.1.1 Task 1(a): Forward Finite Difference Error

Let f be sufficiently smooth near the real point x. The forward finite difference approximation to the first derivative is

$$D_h^+ f(x) = \frac{f(x+h) - f(x)}{h},$$

where $h > 0$ is the step size. By Taylor expansion about x,

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{6}f'''(x) + O(h^4).$$

Subtracting $f(x)$ from both sides gives

$$f(x+h) - f(x) = hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{6}f'''(x) + O(h^4).$$

After division by h, we obtain

$$\frac{f(x+h)-f(x)}{h} = f'(x) + \frac{h}{2}f''(x) + \frac{h^2}{6}f'''(x) + O(h^3).$$

Therefore, the error in the forward finite difference approximation is

$$D_h^+ f(x) - f'(x) = \frac{h}{2}f''(x) + \frac{h^2}{6}f'''(x) + O(h^3).$$

The leading truncation error term is

$$\boxed{\frac{h}{2}f''(x)}$$

and hence

$$D_h^+ f(x) - f'(x) = O(h).$$

This shows that the forward finite difference method is first order accurate with respect to the step size. However, this conclusion only describes truncation error. In floating point arithmetic, the subtraction $f(x+h) - f(x)$ may become unreliable when h is very small.

2.1.2 Round Off Error in the Forward Difference Formula

To understand the effect of floating point arithmetic, suppose the computed function values are

$$\widetilde{f}(x+h) = f(x+h)(1+\delta_1), \qquad \widetilde{f}(x) = f(x)(1+\delta_0),$$

where

$$|\delta_0|, |\delta_1| \leq \epsilon_{\text{mach}}.$$

The computed forward difference is then

$$\widetilde{D}_h^+ f(x) = \frac{\widetilde{f}(x+h) - \widetilde{f}(x)}{h}.$$

Substituting the perturbed values gives

$$\widetilde{D}_h^+ f(x) = \frac{f(x+h)-f(x)}{h} + \frac{\delta_1 f(x+h) - \delta_0 f(x)}{h}.$$

The second term represents the round off contribution. Its magnitude is bounded approximately by

$$\left|\frac{\delta_1 f(x+h) - \delta_0 f(x)}{h}\right| \leq \frac{\epsilon_{\text{mach}}\left(|f(x+h)| + |f(x)|\right)}{h}.$$

For small h, $f(x+h) \approx f(x)$, so the round off contribution behaves like

$$E_{\text{round}} \approx \frac{2\epsilon_{\text{mach}}|f(x)|}{h}.$$

Thus, reducing h decreases the truncation error but can increase the round off error. This is the classical error trade off in finite difference differentiation and is one reason why high order finite difference formulas can become numerically delicate [1].

2.1.3 Task 1(b): Derivation of the Complex Step Approximation

The complex step method evaluates the function at $x + ih$, where $i^2 = -1$. Assume that f is analytic in a neighbourhood of x, and that its derivatives on the real axis are real. Taylor expansion gives

$$f(x+ih) = f(x) + ihf'(x) + \frac{(ih)^2}{2}f''(x) + \frac{(ih)^3}{6}f'''(x) + \frac{(ih)^4}{24}f^{(4)}(x) + O(h^5).$$

Using

$$(ih)^2 = -h^2, \qquad (ih)^3 = -ih^3, \qquad (ih)^4 = h^4,$$

this becomes

$$f(x+ih) = f(x) + ihf'(x) - \frac{h^2}{2}f''(x) - i\frac{h^3}{6}f'''(x) + \frac{h^4}{24}f^{(4)}(x) + O(h^5).$$

Taking the imaginary part gives

$$\mathrm{Im}\,(f(x+ih)) = hf'(x) - \frac{h^3}{6}f'''(x) + O(h^5).$$

Dividing by h, we obtain

$$\frac{\mathrm{Im}\,(f(x+ih))}{h} = f'(x) - \frac{h^2}{6}f'''(x) + O(h^4).$$

Therefore, the complex step derivative approximation is

$$\boxed{f'(x) \approx \frac{\mathrm{Im}\,(f(x+ih))}{h}}.$$

2.1.4 Task 1(c): Truncation Error of the Complex Step Method

From the expansion above, the error of the complex step method is

$$\frac{\mathrm{Im}\,(f(x+ih))}{h} - f'(x) = -\frac{h^2}{6}f'''(x) + O(h^4).$$

The leading truncation error term is therefore

$$\boxed{-\frac{h^2}{6}f'''(x)}$$

and hence

$$\frac{\mathrm{Im}\,(f(x+ih))}{h} - f'(x) = O(h^2).$$

Thus, the complex step approximation has second order truncation error. This is already better than the $O(h)$ truncation error of the forward finite difference method.

2.1.5 Task 1(d): Absence of Subtractive Cancellation

The main numerical advantage of the complex step method is not only its $O(h^2)$ truncation error, but also its avoidance of subtractive cancellation. The forward finite difference method computes

$$f(x+h) - f(x).$$

When h is small, the two values $f(x+h)$ and $f(x)$ are very close. Their subtraction can lose significant digits, and the resulting small difference is then divided by h, which further amplifies round off error.

In contrast, the complex step method computes

$$\frac{\operatorname{Im}\left(f(x+ih)\right)}{h}.$$

From the Taylor expansion,

$$\operatorname{Im}\left(f(x+ih)\right) = hf'(x) - \frac{h^3}{6}f'''(x) + O(h^5).$$

The derivative information is contained directly in the imaginary part of a single complex function evaluation. The real component $f(x)$, which may be much larger than the derivative perturbation, remains in the real part and is not subtracted from another nearly equal real number.

As a result, the complex step method does not contain the subtraction driven round off term of size approximately

$$\frac{\epsilon_{\text{mach}}|f(x)|}{h}.$$

This explains why the method can remain accurate for very small step sizes, provided that the function is analytic and can be evaluated reliably for complex arguments. This property is the key reason why the complex step method is often much more stable than real finite difference formulas for first derivative approximation.

2.2 Task 2: First Derivative Error Comparison

Task 2 compares the numerical behaviour of the forward finite difference method and the complex step method for a first derivative problem. The test function is chosen as

$$f(x) = \sin x,$$

and the derivative is evaluated at

$$x_0 = \frac{\pi}{4}.$$

The exact derivative is

$$f'(x_0) = \cos\left(\frac{\pi}{4}\right) = \frac{\sqrt{2}}{2} \approx 0.7071067811865476.$$

This exact value is used as the reference solution for computing the numerical errors.

The step sizes are generated as 100 logarithmically spaced values in the interval

$$h \in [10^{-16}, 10^{-1}].$$

This range is deliberately wide. Relatively large step sizes show the effect of truncation error, while very small step sizes show the effect of round off error and subtractive cancellation.

For each value of h, the forward finite difference approximation is computed by

$$D_h^+ f(x_0) = \frac{f(x_0 + h) - f(x_0)}{h}.$$

The corresponding absolute error is

$$E_{\mathrm{FD}}(h) = \left| D_h^+ f(x_0) - f'(x_0) \right|.$$

The empirical optimal step size for the forward finite difference method is defined as

$$h_{\mathrm{opt}} = \arg\min_h E_{\mathrm{FD}}(h).$$

The complex step approximation is computed using

$$D_h^{\mathrm{CS}} f(x_0) = \frac{\mathrm{Im}\left(f(x_0 + ih)\right)}{h}.$$

Its absolute error is defined as

$$E_{\mathrm{CS}}(h) = \left| D_h^{\mathrm{CS}} f(x_0) - f'(x_0) \right|.$$

The same set of step sizes is used for both methods, so the two error curves can be compared directly.

In the MATLAB implementation, all approximations are computed using vectorised array operations. The two absolute error curves are plotted on a log log scale, which allows the behaviour over many orders of magnitude to be observed clearly. Since logarithmic plots cannot display zero values, any zero displayed complex step errors are replaced by machine epsilon only for plotting. The numerical results reported in the table are still based on the original MATLAB output.

2.3 Task 3: FFT Based Cauchy Differentiation

Task 3 computes high order derivatives of an analytic function by combining Cauchy's integral formula with the fast Fourier transform. For a function f analytic inside and on a closed contour C containing z_0, Cauchy's integral formula gives

$$f^{(n)}(z_0) = \frac{n!}{2\pi i} \oint_C \frac{f(\zeta)}{(\zeta - z_0)^{n+1}}\, d\zeta.$$

In this task, the contour is chosen as a circle centred at z_0 with radius r:

$$\zeta(\theta) = z_0 + re^{i\theta}, \qquad 0 \le \theta < 2\pi.$$

Since

$$d\zeta = ire^{i\theta}\, d\theta,$$

substitution into Cauchy's formula gives

$$f^{(n)}(z_0) = \frac{n!}{2\pi r^n} \int_0^{2\pi} f(z_0 + re^{i\theta}) e^{-in\theta}\, d\theta.$$

This integral is the n-th Fourier coefficient of the function values sampled on the circular contour, multiplied by $n!/r^n$.

To approximate the integral numerically, N equally spaced points are used:

$$\theta_k = \frac{2\pi k}{N}, \qquad k = 0, 1, \ldots, N-1.$$

The corresponding contour points are

$$z_k = z_0 + re^{2\pi ik/N}.$$

Applying the trapezoidal rule gives

$$f^{(n)}(z_0) \approx \frac{n!}{Nr^n} \sum_{k=0}^{N-1} f(z_k) e^{-2\pi ikn/N}.$$

The summation is the n-th discrete Fourier coefficient of the sampled values $f(z_k)$. Therefore, the derivative can be approximated by

$$f^{(n)}(z_0) \approx \frac{n!}{r^n} \widehat{f}_n,$$

where

$$\widehat{f}_n = \frac{1}{N} \sum_{k=0}^{N-1} f(z_k) e^{-2\pi ikn/N}.$$

In MATLAB, the command `fft` computes the unnormalised discrete Fourier transform. Therefore, the Fourier coefficients must be computed using

$$\widehat{f}_n = \frac{\texttt{fft}(f(z_k))_{n+1}}{N},$$

where the index $n+1$ is used because MATLAB arrays start from index 1. The derivative approximation implemented in MATLAB is therefore

$$f^{(n)}(z_0) \approx \frac{n!}{r^n} \frac{\texttt{fft}(f(z_k))_{n+1}}{N}.$$

This scaling is essential. Without the factor $1/N$, the computed derivatives would be incorrectly scaled.

The coursework function used in this task is

$$f(z) = \frac{e^z}{\sin^3(z) + \cos^3(z)}.$$

The fifth derivative at the origin is known to be

$$f^{(5)}(0) = -164.$$

The main calculation uses

$$z_0 = 0, \qquad r = 0.4, \qquad N = 32.$$

To investigate convergence, the same derivative is also computed for

$$N = 16,\ 32,\ 64,\ 128.$$

In addition, the effect of the contour radius is tested by comparing $r = 0.3$ with $r = 0.4$ while keeping $N = 32$.

Before applying the method to the coursework function, the implementation is tested on

$$f(z) = e^z.$$

Since all derivatives of e^z at $z = 0$ are equal to 1, this test verifies that the FFT coefficient scaling and derivative conversion have been implemented correctly. The FFT based Cauchy method is particularly suitable for analytic functions, and its connection with Fourier and spectral methods is consistent with classical work on numerical differentiation of analytic functions [3, 4].

2.4 Task 4: Central Finite Difference Stencil for the Fifth Derivative

Task 4 investigates the stability of a real axis finite difference formula for a high order derivative. The same analytic function used in Task 3 is restricted to the real axis. For real x, define

$$g(x) = f(x) = \frac{e^x}{\sin^3(x) + \cos^3(x)}.$$

The exact fifth derivative at the origin is

$$g^{(5)}(0) = -164.$$

This exact value is used as the reference solution for measuring the accuracy of the finite difference approximation.

The central finite difference stencil used to approximate the fifth derivative is

$$g^{(5)}(0) \approx \frac{-g(-3h) + 4g(-2h) - 5g(-h) + 5g(h) - 4g(2h) + g(3h)}{2h^5}.$$

This formula uses function values symmetrically located around the point $x = 0$. Its truncation error is formally of order $O(h^2)$. Therefore, if only truncation error were present, reducing h would be expected to improve the approximation.

However, the denominator contains h^5, which makes the formula highly sensitive to floating point error when h is small. The numerator is a linear combination of several nearby function values. For small h, these function values are very close to each other, and the alternating coefficients in the stencil produce substantial cancellation. Any small round off perturbation in the numerator is then divided by $2h^5$, so the resulting error can be greatly amplified.

The MATLAB experiment evaluates this stencil for logarithmically spaced step sizes in the interval

$$h \in [10^{-12}, 10^{-1}].$$

For each step size, the finite difference approximation is computed and compared with the exact value -164. The absolute error is defined as

$$E_{\mathrm{FD5}}(h) = \left| g^{(5)}_{\mathrm{FD}}(0; h) - (-164) \right|,$$

where $g^{(5)}_{\mathrm{FD}}(0; h)$ denotes the fifth derivative approximation obtained from the central finite difference stencil with step size h.

The purpose of this experiment is to show that a high order finite difference stencil may have a good formal truncation error but still perform poorly in floating point arithmetic. For large h, truncation error dominates because the stencil is only a local approximation to the derivative. For very small h, round off error dominates because cancellation in the numerator is magnified by the factor h^{-5}. Therefore, the error curve is expected to have a narrow region where the two error sources are balanced. This behaviour also illustrates the practical difficulty of using high order finite difference formulas, which are closely related to ill conditioned interpolation and Vandermonde type systems [1].

3 Numerical Results and Analysis

3.1 Task 2: Forward Difference and Complex Step Comparison

The first numerical experiment compares the forward finite difference method with the complex step method for

$$f(x) = \sin x$$

at

$$x = \frac{\pi}{4}.$$

The exact derivative is

$$f'\left(\frac{\pi}{4}\right) = \cos\left(\frac{\pi}{4}\right) = \frac{\sqrt{2}}{2} \approx 0.7071067811865476.$$

Both methods were evaluated using 100 logarithmically spaced step sizes over the interval

$$h \in [10^{-16}, 10^{-1}].$$

The absolute errors were then plotted against h on a log log scale.

Figure 1 shows the error behaviour of the two methods. The forward finite difference method has a clear trade off curve. When h is relatively large, the error is mainly caused by truncation error, so decreasing h improves the approximation. However, when h becomes too small, the subtraction $f(x + h) - f(x)$ involves two nearly equal numbers. This causes subtractive cancellation, and the remaining error is amplified by division by h. As a result, the forward finite difference error starts to increase again for very small step sizes.

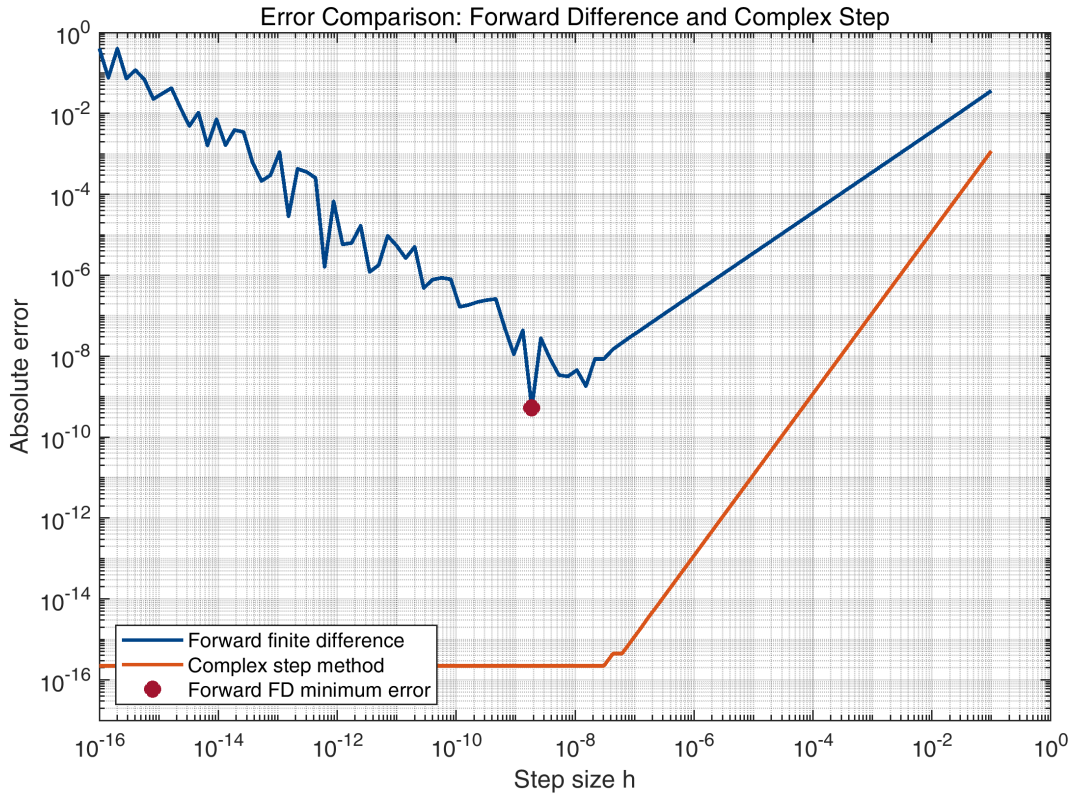

Figure 1: Absolute error comparison between the forward finite difference method and the complex step method for $f(x) = \sin x$ at $x = \pi/4$.

The numerical results are summarised in Table 1. The forward finite difference method reaches its minimum absolute error at

$$h_{\text{opt}} = 1.874 \times 10^{-9}.$$

The corresponding minimum absolute error is

$$5.287 \times 10^{-10}.$$

This value of h_{opt} is consistent with the theoretical expectation that the best step size for a first order finite difference method is close to $\sqrt{\epsilon_{\text{mach}}}$.

Table 1: Minimum absolute errors for the forward finite difference and complex step methods.

Method	Step size at minimum error	Minimum absolute error
Forward finite difference	1.874×10^{-9}	5.287×10^{-10}
Complex step method	1.000×10^{-16}	displayed as 0

The complex step method behaves very differently. Its error does not show the same rapid increase at small h. In the MATLAB output, the minimum absolute error is displayed as zero, meaning that the computed derivative agrees with the exact derivative to the displayed double precision accuracy. This does not mean that the mathematical error is exactly zero. Rather, it shows that the error is below the displayed numerical precision of the computation.

For the logarithmic plot in Figure 1, zero displayed complex step errors were replaced by machine epsilon only for plotting, since a logarithmic scale cannot display zero values. This adjustment affects only the visualisation and not the numerical results reported in Table 1.

Overall, this experiment confirms the theoretical analysis in Task 1. The forward finite difference method is limited by a balance between truncation error and round off error. The complex step method avoids the subtraction of nearly equal real numbers, so it remains stable even when h is extremely small. This explains why the complex step method gives a much more reliable first derivative approximation in this example.

3.2 Task 3: High Order Derivatives from the FFT Based Cauchy Method

The FFT based Cauchy method was then applied to the analytic function

$$f(z) = \frac{e^z}{\sin^3(z) + \cos^3(z)}.$$

The goal is to compute the fifth derivative at the origin. The exact value is

$$f^{(5)}(0) = -164.$$

The main computation uses the circular contour centred at

$$z_0 = 0,$$

with radius

$$r = 0.4$$

and

$$N = 32$$

equally spaced sample points.

Before applying the method to the coursework function, the implementation was tested using

$$f(z) = e^z.$$

This is a useful validation test because every derivative of e^z at $z = 0$ is equal to 1. The computed derivatives from order 0 to order 5 agreed with the exact value to approximately machine precision. The largest absolute error in this validation test was

$$7.2294 \times 10^{-14}.$$

This confirms that the Fourier coefficient scaling and the conversion from FFT coefficients to derivatives were implemented correctly.

For the coursework function, the main computation with $r = 0.4$ and $N = 32$ gave

$$f^{(5)}(0) \approx -164.0000000461220679.$$

The absolute error was therefore

$$4.6122 \times 10^{-8}.$$

This is already a good approximation to the exact value -164, but the accuracy can be improved by increasing the number of contour sample points.

Table 2 shows the accuracy of the FFT based Cauchy method for different values of N, while keeping $r = 0.4$ fixed. The error decreases rapidly as N increases from 16 to 64. In particular, the error falls from 2.2142×10^{-3} at $N = 16$ to 1.1369×10^{-13} at $N = 64$. Increasing N further to 128 does not reduce the error, which suggests that the computation has already reached the floating point accuracy limit.

Table 2: Accuracy of the FFT based Cauchy method for different values of N with $r = 0.4$.

N	Approximation of $f^{(5)}(0)$	Absolute error
16	-164.0022142099	2.2142×10^{-3}
32	-164.0000000461	4.6122×10^{-8}
64	-164.0000000000	1.1369×10^{-13}
128	-164.0000000000	1.1369×10^{-13}

The same convergence behaviour is illustrated in Figure 2. The error decreases very quickly as the number of sample points increases. This is consistent with the high accuracy expected from Fourier based methods when they are applied to analytic functions. The flattening of the curve for larger N indicates that round off error and floating point precision have become the main limitations.

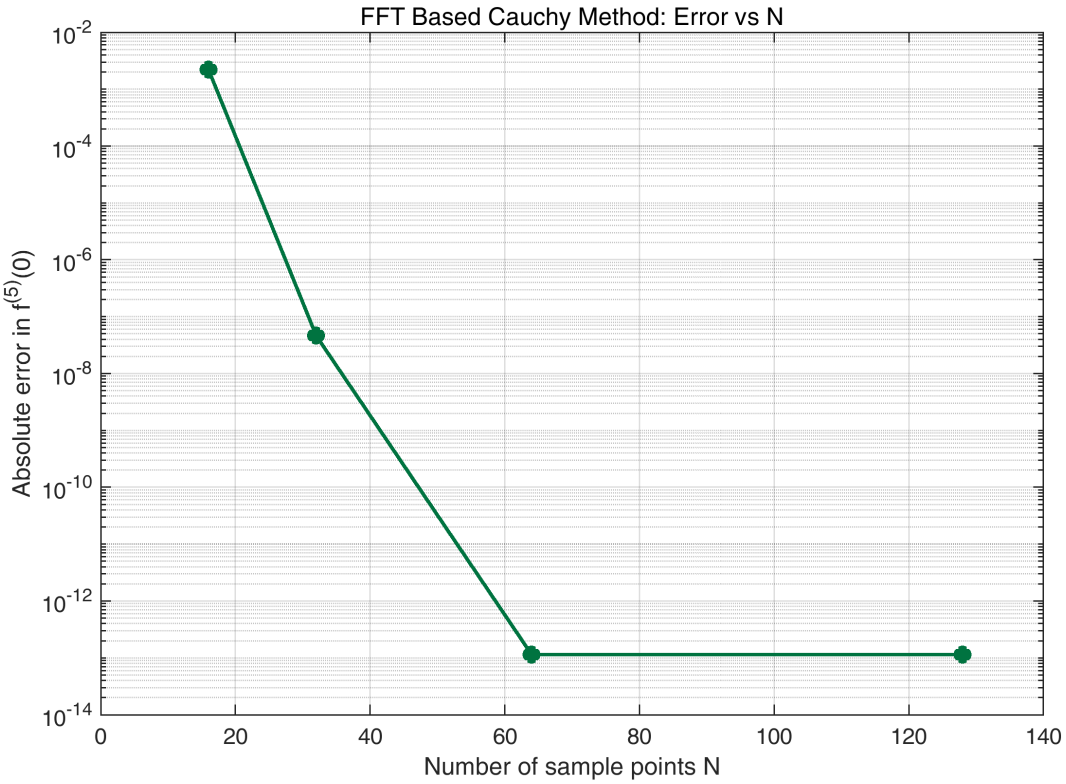

Figure 2: Absolute error of the FFT based Cauchy method for computing $f^{(5)}(0)$ as the number of sample points N increases.

The effect of the contour radius was also tested. Table 3 compares the results for

$r = 0.3$ and $r = 0.4$, both with $N = 32$. In this experiment, $r = 0.3$ gives a smaller error than $r = 0.4$. The error for $r = 0.3$ is

$$1.9611 \times 10^{-12},$$

whereas the error for $r = 0.4$ is

$$4.6122 \times 10^{-8}.$$

Table 3: Effect of the contour radius on the FFT based approximation with $N = 32$.

Radius r	Approximation of $f^{(5)}(0)$	Absolute error
0.3	-164.0000000000	1.9611×10^{-12}
0.4	-164.0000000461	4.6122×10^{-8}

This comparison shows that the radius of the circular contour is an important numerical parameter. If the radius is too large, the contour may approach regions where the analytic function varies more strongly, or it may move closer to singularities outside the safe analytic region. This can reduce accuracy for a fixed number of points. If the radius is too small, the factor r^{-n} in the derivative formula can amplify round off error, especially for high order derivatives. Therefore, the contour radius must balance discretisation error and round off error.

Overall, the FFT based Cauchy method performs very well for this analytic test function. The results show rapid convergence as N increases, and the method reaches an error close to machine precision for $N = 64$. This is much more accurate than the high order finite difference result examined in Task 4, and it demonstrates the advantage of using complex contour information for high order numerical differentiation.

3.3 Task 4: Failure of the Fifth Derivative Finite Difference Stencil

The final numerical experiment applies the central finite difference stencil to approximate the fifth derivative of the real axis function

$$g(x) = \frac{e^x}{\sin^3(x) + \cos^3(x)}.$$

The exact value used for comparison is

$$g^{(5)}(0) = -164.$$

The stencil was evaluated over logarithmically spaced step sizes in the interval

$$h \in [10^{-12}, 10^{-1}].$$

For each value of h, the fifth derivative approximation and the absolute error were computed.

The best finite difference result occurred at

$$h = 2.683 \times 10^{-3}.$$

At this step size, the approximation was

$$g^{(5)}(0) \approx -164.0330873784,$$

with minimum absolute error

$$3.309 \times 10^{-2}.$$

Although this is the best result obtained by the finite difference stencil in this experiment, it is much less accurate than the FFT based Cauchy method in Task 3. In particular, the FFT based method achieved an error of approximately

$$1.1369 \times 10^{-13}$$

when $N = 64$ and $r = 0.4$. This comparison shows that the finite difference stencil is much more sensitive to numerical error when it is used for a high order derivative.

Table 4: Best result obtained by the central finite difference stencil for the fifth derivative.

Best step size	Best approximation	Minimum absolute error
2.683×10^{-3}	-164.0330873784	3.309×10^{-2}

Table 5 gives selected results for representative step sizes. The results show that the finite difference approximation is extremely unstable when h is very small. For example, at

$$h = 10^{-12},$$

the absolute error is approximately

$$3.3307 \times 10^{44}.$$

This very large error is not caused by the mathematical derivative itself, but by floating point round off error being magnified by the denominator $2h^5$. Since h^5 is extremely small, even a tiny perturbation in the numerator can become enormous after division.

Table 5: Selected finite difference results for different step sizes.

Step size h	Approximation	Absolute error
10^{-12}	3.3307×10^{44}	3.3307×10^{44}
10^{-10}	0	164
10^{-8}	1.1102×10^{24}	1.1102×10^{24}
10^{-6}	-2.2204×10^{14}	2.2204×10^{14}
10^{-4}	3.3307×10^{4}	3.3471×10^{4}
10^{-2}	-164.4463	4.4629×10^{-1}
10^{-1}	-213.3478	4.9348×10^{1}

The error curve is shown in Figure 3. The behaviour is similar to the classical finite difference trade off, but the instability is much more severe because a fifth derivative is being approximated. For very small h, round off error dominates. The stencil combines several nearby function values with alternating coefficients, so there is strong cancellation in the numerator. The result is then divided by $2h^5$, which greatly amplifies any

floating point perturbation. For larger h, the round off error becomes less dominant, but truncation error increases because the finite difference stencil is no longer a sufficiently accurate local approximation.

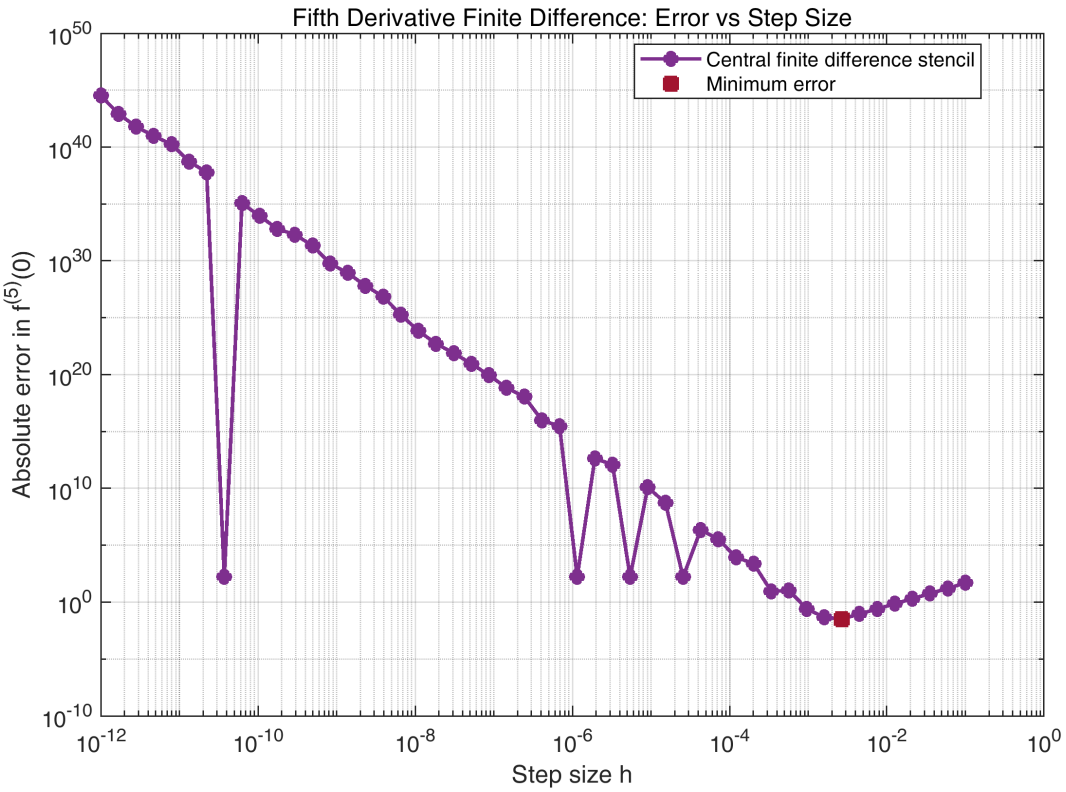

Figure 3: Absolute error of the central finite difference stencil for approximating $g^{(5)}(0)$.

This experiment explains why high order finite difference formulas can be unreliable in practice. Although the stencil has formal truncation error $O(h^2)$, this theoretical order does not guarantee high numerical accuracy in floating point arithmetic. The practical error is strongly affected by cancellation, division by a high power of h, and the conditioning of the stencil coefficients. High order finite difference formulas are closely related to interpolation and Vandermonde type systems, which can become ill conditioned as the derivative order increases [1]. Therefore, small perturbations in sampled function values may be amplified significantly.

Compared with the FFT based Cauchy method, the finite difference stencil is much less stable. The FFT based method uses function values on a complex contour and extracts derivative information through Fourier coefficients. It reached an error close to 10^{-13} for the fifth derivative test. By contrast, the best finite difference error remained around 10^{-2}. This confirms that, for analytic functions and high order derivatives, the complex contour approach is far more accurate and stable than a real axis finite difference stencil.

4 Concluding Remarks

This report has examined several numerical differentiation methods from both theoretical and computational perspectives. The forward finite difference method was first analysed using Taylor series expansion. Its leading truncation error was shown to be $O(h)$, while

its round off error increases when the step size becomes too small. This explains the typical error trade off observed in the Task 2 experiment. For $f(x) = \sin x$ at $x = \pi/4$, the forward finite difference method achieved its best result at

$$h = 1.874 \times 10^{-9},$$

with minimum absolute error

$$5.287 \times 10^{-10}.$$

This confirms that the step size cannot be chosen arbitrarily small in a real finite difference calculation.

The complex step method gave much more stable first derivative approximations. Its Taylor expansion shows that the truncation error is $O(h^2)$, and the method avoids the direct subtraction of two nearly equal real numbers. In the Task 2 experiment, the complex step error was displayed as zero in MATLAB output, meaning that the computed derivative agreed with the exact value to the displayed double precision accuracy. This result supports the theoretical conclusion that the complex step method is far less affected by subtractive cancellation than the forward finite difference method.

For high order derivatives, the FFT based Cauchy method provided the most accurate results in this report. By sampling the analytic function on a circular contour and using Fourier coefficients, the method computed the fifth derivative of

$$f(z) = \frac{e^z}{\sin^3(z) + \cos^3(z)}$$

with very high accuracy. When $r = 0.4$, increasing N from 16 to 64 reduced the absolute error from

$$2.2142 \times 10^{-3}$$

to

$$1.1369 \times 10^{-13}.$$

Increasing N further to 128 did not improve the result, which indicates that the computation had essentially reached the floating point accuracy limit. The radius comparison also showed that the contour radius is an important numerical parameter, since $r = 0.3$ gave a smaller error than $r = 0.4$ when $N = 32$.

In contrast, the fifth derivative finite difference stencil in Task 4 was much less stable. Although the stencil has formal truncation error $O(h^2)$, its practical accuracy was strongly limited by round off error and cancellation. The denominator $2h^5$ greatly amplifies small perturbations in the numerator when h is small. The best finite difference result occurred at

$$h = 2.683 \times 10^{-3},$$

with approximation

$$g^{(5)}(0) \approx -164.0330873784$$

and minimum absolute error

$$3.309 \times 10^{-2}.$$

This is much less accurate than the FFT based Cauchy result, which reached an error close to 10^{-13}.

Overall, the experiments show that finite difference methods are simple and useful, but their accuracy depends strongly on the choice of step size. The complex step method is

a substantial improvement for first derivatives because it avoids subtractive cancellation. For analytic functions and high order derivatives, the FFT based Cauchy method is the most accurate and stable method among those tested in this report. These results demonstrate that the best numerical differentiation method depends not only on the derivative order, but also on the analytic structure of the function and the limitations of floating point arithmetic.

5 AI Usage Statement

During the preparation of this coursework, I used ChatGPT as a supplementary learning support tool. It was mainly used to help me clarify difficult concepts in numerical differentiation, including truncation error, subtractive cancellation, the complex step method, and the FFT based Cauchy method. It was also used to help organise the report structure and improve the clarity of some explanations, equations, tables, and figure captions.

The MATLAB code used in this coursework was written by myself. I used AI only to discuss the general logic of the numerical methods, check whether my understanding of the algorithms was reasonable, and clarify issues related to LaTeX formatting and report presentation. All scripts were run and tested by myself in MATLAB. The numerical results, figures, tables, and final interpretation were checked against the coursework requirements by myself.

Therefore, AI was used as a learning assistant and editing aid, rather than as a replacement for independent work. The submitted report reflects my own understanding of the mathematical derivations, MATLAB implementation, numerical experiments, and conclusions.

References

[1] Fornberg, B. (1981). Numerical Differentiation of Analytic Functions. *ACM Transactions on Mathematical Software*, 7(4), 512–526.

[2] Martins, J. R. R. A., Sturdza, P., & Alonso, J. J. (2003). The Complex-Step Derivative Approximation. *ACM Transactions on Mathematical Software*, 29(3), 245–262.

[3] Lyness, J. N., & Moler, C. B. (1967). Numerical Differentiation of Analytic Functions. *SIAM Journal on Numerical Analysis*, 4(2), 202–210.

[4] Trefethen, L. N. (2000). *Spectral Methods in MATLAB*. SIAM.

A MATLAB Code

The MATLAB code used for the numerical experiments is included in this appendix. The main script runs the experiments for Task 2, Task 3, and Task 4. Two function files are used for the FFT based Cauchy derivative method and the fifth derivative finite difference stencil.

A.1 Main Script

Listing 1: Main MATLAB script for Task 2, Task 3, and Task 4.

```
%% MTH208 Coursework Project 2
% Numerical Differentiation Beyond the Real Plane
% Main script for Task 2, Task 3, and Task 4

clear; close all; clc;
format long e;

%% Create figure folder

figDir = fullfile(pwd, 'figures');

if ~isfolder(figDir)
    mkdir(figDir);
end

%% Task 2: Error Trade Off Simulation

fprintf('Task 2: Error Trade Off Simulation\n');
fprintf('----------------------------------\n');

% Test function and exact derivative
f = @(x) sin(x);
x0 = pi/4;
exact_derivative = cos(x0);

% Generate 100 logarithmically spaced step sizes
h_vals = logspace(-16, -1, 100);

if any(h_vals <= 0)
    error('All step sizes must be positive.');
end

% Forward finite difference approximation
fd_approx = (f(x0 + h_vals) - f(x0)) ./ h_vals;
fd_error = abs(fd_approx - exact_derivative);

% Complex step derivative approximation
cs_approx = imag(f(x0 + 1i*h_vals)) ./ h_vals;
cs_error = abs(cs_approx - exact_derivative);

% Identify minimum errors and step sizes
[min_fd_error, idx_fd] = min(fd_error);
hopt_fd = h_vals(idx_fd);

[min_cs_error, idx_cs] = min(cs_error);
hopt_cs = h_vals(idx_cs);

% Print results
fprintf('Exact derivative cos(pi/4): %.16f\n\n', exact_derivative);

fprintf('Forward finite difference:\n');
fprintf('Optimal h: %.3e\n', hopt_fd);
fprintf('Minimum absolute error: %.3e\n\n', min_fd_error);

fprintf('Complex step method:\n');
fprintf('Step size at minimum error: %.3e\n', hopt_cs);
fprintf('Minimum absolute error: %.3e\n\n', min_cs_error);

task2_summary = table( ...
    {'Forward finite difference'; 'Complex step method'}, ...
    [hopt_fd; hopt_cs], ...
    [min_fd_error; min_cs_error], ...
    'VariableNames', {'Method', 'StepSizeAtMinimumError', 'MinimumAbsoluteError'} ...
);

disp(task2_summary);

%% Plot Task 2

```

```
% Use eps only for plotting because log scale cannot display zero.
fd_error_plot = max(fd_error, eps);
cs_error_plot = max(cs_error, eps);

figure('Color', 'w', 'Position', [100, 100, 800, 550]);

loglog(h_vals, fd_error_plot, ...
    'Color', [0.000, 0.267, 0.533], ...
    'LineWidth', 1.8);
hold on;

loglog(h_vals, cs_error_plot, ...
    'Color', [0.850, 0.325, 0.098], ...
    'LineWidth', 1.8);

loglog(hopt_fd, max(min_fd_error, eps), 'o', ...
    'Color', [0.635, 0.078, 0.184], ...
    'MarkerFaceColor', [0.635, 0.078, 0.184], ...
    'MarkerSize', 7, ...
    'LineWidth', 1.5);

xlabel('Step size h', 'FontSize', 12);
ylabel('Absolute error', 'FontSize', 12);
title('Error Comparison: Forward Difference and Complex Step', 'FontSize', 13);

legend( ...
    'Forward finite difference', ...
    'Complex step method', ...
    'Forward FD minimum error', ...
    'Location', 'southwest' ...
);

grid on;
set(gca, 'FontSize', 11);
ylim([1e-17, 1e0]);

pngFile_task2 = fullfile(figDir, 'fig_task2_error_comparison.png');
drawnow;
exportgraphics(gcf, pngFile_task2, 'Resolution', 300);


%% Task 3: High Order Derivatives via FFT

fprintf('\nTask 3: High Order Derivatives via FFT\n');
fprintf('--------------------------------------\n');

% Test the FFT derivative solver using f(z) = exp(z)
% All derivatives of exp(z) at z0 = 0 are equal to 1.

f_test = @(z) exp(z);
z0 = 0;
r_test = 0.4;
N_test = 32;
nmax_test = 5;

derivs_test = cauchy_fft_derivatives(f_test, z0, r_test, N_test, nmax_test);
exact_test = ones(nmax_test + 1, 1);
test_errors = abs(derivs_test - exact_test);

test_table = table( ...
    (0:nmax_test)', ...
    real(derivs_test), ...
    imag(derivs_test), ...
    test_errors, ...
    'VariableNames', {'DerivativeOrder', 'RealApproximation', 'ImaginaryPart', '
        AbsoluteError'} ...
);

fprintf('\nTest function f(z) = exp(z):\n');
disp(test_table);

% Coursework function
f_analytic = @(z) exp(z) ./ (sin(z).^3 + cos(z).^3);
```

```
142
143 exact_fifth = -164;
144 n_order = 5;
145
146 % Main calculation with r = 0.4 and N = 32
147 r = 0.4;
148 N = 32;
149
150 derivs_main = cauchy_fft_derivatives(f_analytic, z0, r, N, n_order);
151 approx_fifth_main = derivs_main(n_order + 1);
152 error_fifth_main = abs(real(approx_fifth_main) - exact_fifth);
153
154 fprintf('\nMain calculation for coursework function:\n');
155 fprintf('Radius r: %.2f\n', r);
156 fprintf('Number of points N: %d\n', N);
157 fprintf('Approximation of f^(5)(0): %.16f\n', real(approx_fifth_main));
158 fprintf('Imaginary part: %.3e\n', imag(approx_fifth_main));
159 fprintf('Exact value: %.16f\n', exact_fifth);
160 fprintf('Absolute error: %.3e\n', error_fifth_main);
161
162 % Accuracy dependence on N
163 N_values = [16, 32, 64, 128];
164
165 approx_N = zeros(size(N_values));
166 imag_N = zeros(size(N_values));
167 error_N = zeros(size(N_values));
168
169 for k = 1:length(N_values)
170     current_N = N_values(k);
171     derivs_N = cauchy_fft_derivatives(f_analytic, z0, r, current_N, n_order);
172     approx_N(k) = real(derivs_N(n_order + 1));
173     imag_N(k) = imag(derivs_N(n_order + 1));
174     error_N(k) = abs(approx_N(k) - exact_fifth);
175 end
176
177 N_table = table( ...
178     N_values', ...
179     approx_N', ...
180     imag_N', ...
181     error_N', ...
182     'VariableNames', {'N', 'Approximation', 'ImaginaryPart', 'AbsoluteError'} ...
183 );
184
185 fprintf('\nAccuracy dependence on N with r = 0.4:\n');
186 disp(N_table);
187
188 % Effect of radius
189 r_values = [0.3, 0.4];
190 N_radius = 32;
191
192 approx_r = zeros(size(r_values));
193 imag_r = zeros(size(r_values));
194 error_r = zeros(size(r_values));
195
196 for k = 1:length(r_values)
197     current_r = r_values(k);
198     derivs_r = cauchy_fft_derivatives(f_analytic, z0, current_r, N_radius, n_order);
199     approx_r(k) = real(derivs_r(n_order + 1));
200     imag_r(k) = imag(derivs_r(n_order + 1));
201     error_r(k) = abs(approx_r(k) - exact_fifth);
202 end
203
204 r_table = table( ...
205     r_values', ...
206     approx_r', ...
207     imag_r', ...
208     error_r', ...
209     'VariableNames', {'Radius', 'Approximation', 'ImaginaryPart', 'AbsoluteError'} ...
210 );
211
212 fprintf('\nEffect of radius with N = 32:\n');
213 disp(r_table);
214
```

```
%% Plot Task 3

figure('Color', 'w', 'Position', [100, 100, 800, 550]);

semilogy(N_values, error_N, 'o-', ...
    'Color', [0.000, 0.450, 0.250], ...
    'MarkerFaceColor', [0.000, 0.450, 0.250], ...
    'LineWidth', 1.8, ...
    'MarkerSize', 7);

xlabel('Number of sample points N', 'FontSize', 12);
ylabel('Absolute error in f^{(5)}(0)', 'FontSize', 12);
title('FFT Based Cauchy Method: Error vs N', 'FontSize', 13);

grid on;
set(gca, 'FontSize', 11);

pngFile_task3 = fullfile(figDir, 'fig_task3_error_vs_N.png');
drawnow;
exportgraphics(gcf, pngFile_task3, 'Resolution', 300);


%% Task 4: High Order Finite Difference Failure

fprintf('\nTask 4: High Order Finite Difference Failure\n');
fprintf('-------------------------------------------\n');

% Real axis version of the coursework function
f_real = @(x) real(f_analytic(x));

x0_fd = 0;
exact_fifth = -164;

% Step sizes
h_fd_vals = logspace(-12, -1, 50);

% Fifth derivative approximations and errors
fd5_approx = fifth_deriv_fd(f_real, x0_fd, h_fd_vals);
fd5_error = abs(fd5_approx - exact_fifth);

% Best finite difference result
[min_fd5_error, idx_fd5] = min(fd5_error);
hopt_fd5 = h_fd_vals(idx_fd5);
best_fd5_approx = fd5_approx(idx_fd5);

fprintf('Best step size h: %.3e\n', hopt_fd5);
fprintf('Best approximation of f^(5)(0): %.16f\n', best_fd5_approx);
fprintf('Minimum absolute error: %.3e\n', min_fd5_error);

task4_summary = table( ...
    hopt_fd5, ...
    best_fd5_approx, ...
    min_fd5_error, ...
    'VariableNames', {'BestStepSize', 'BestApproximation', 'MinimumAbsoluteError'} ...
);

fprintf('\nTask 4 summary:\n');
disp(task4_summary);

% Selected step sizes for report table
selected_h = [1e-12, 1e-10, 1e-8, 1e-6, 1e-4, 1e-2, 1e-1];

selected_approx = fifth_deriv_fd(f_real, x0_fd, selected_h);
selected_error = abs(selected_approx - exact_fifth);

task4_selected_table = table( ...
    selected_h', ...
    selected_approx', ...
    selected_error', ...
    'VariableNames', {'StepSize', 'Approximation', 'AbsoluteError'} ...
);

fprintf('\nSelected Task 4 finite difference results:\n');
```

```
disp(task4_selected_table);

%% Plot Task 4

figure('Color', 'w', 'Position', [100, 100, 800, 550]);

loglog(h_fd_vals, fd5_error, 'o-', ...
    'Color', [0.494, 0.184, 0.556], ...
    'MarkerFaceColor', [0.494, 0.184, 0.556], ...
    'LineWidth', 1.8, ...
    'MarkerSize', 6);
hold on;

loglog(hopt_fd5, min_fd5_error, 's', ...
    'Color', [0.635, 0.078, 0.184], ...
    'MarkerFaceColor', [0.635, 0.078, 0.184], ...
    'MarkerSize', 8, ...
    'LineWidth', 1.6);

xlabel('Step size h', 'FontSize', 12);
ylabel('Absolute error in f^{(5)}(0)', 'FontSize', 12);
title('Fifth Derivative Finite Difference: Error vs Step Size', 'FontSize', 13);

legend( ...
    'Central finite difference stencil', ...
    'Minimum error', ...
    'Location', 'best' ...
);

grid on;
set(gca, 'FontSize', 11);

pngFile_task4 = fullfile(figDir, 'fig_task4_fd5_error.png');
drawnow;
exportgraphics(gcf, pngFile_task4, 'Resolution', 300);
```

A.2 FFT Based Cauchy Derivative Function

Listing 2: MATLAB function for computing derivatives using Cauchy's integral formula and the FFT.

```
function derivs = cauchy_fft_derivatives(f, z0, r, N, max_deriv)
%CAUCHY_FFT_DERIVATIVES Compute derivatives using Cauchy's integral formula.
%
%   derivs = CAUCHY_FFT_DERIVATIVES(f, z0, r, N, max_deriv)
%   computes approximations to f^(n)(z0), for n = 0, 1, ..., max_deriv.
%   The method samples the analytic function on a circular contour and
%   extracts Fourier coefficients using MATLAB's fft function.
%
%   Inputs:
%       f         - function handle for an analytic function
%       z0        - centre of the circular contour
%       r         - radius of the circular contour
%       N         - number of equally spaced sample points
%       max_deriv - maximum derivative order to compute
%
%   Output:
%       derivs    - column vector where derivs(n+1) approximates f^(n)(z0)
%
%   Method:
%       If z_k = z0 + r*exp(2*pi*i*k/N), then the nth Fourier coefficient
%       of f(z_k) is used to approximate the nth derivative:
%
%           f^(n)(z0) = n! / r^n * fhat_n.
%
%       MATLAB's fft function does not include the 1/N factor, so the
%       Fourier coefficients are computed using fft(fz)/N.

    %% Input checks
```

```
29
30      if ~isa(f, 'function_handle')
31          error('Input f must be a function handle.');
32      end
33
34      if ~(isscalar(r) && isnumeric(r) && r > 0)
35          error('Radius r must be a positive scalar.');
36      end
37
38      if ~(isscalar(N) && isnumeric(N) && N == round(N) && N > 0)
39          error('N must be a positive integer.');
40      end
41
42      if ~(isscalar(max_deriv) && isnumeric(max_deriv) && ...
43              max_deriv == round(max_deriv) && max_deriv >= 0)
44          error('max_deriv must be a non-negative integer.');
45      end
46
47      if max_deriv >= N
48          error('max_deriv must be smaller than N.');
49      end
50
51      %% Sample the function on a circular contour
52
53      k = (0:N-1).';
54      theta = 2*pi*k/N;
55      z = z0 + r*exp(1i*theta);
56
57      fz = f(z);
58
59      if any(~isfinite(fz))
60          error('Function evaluation produced Inf or NaN values.');
61      end
62
63      %% Compute Fourier coefficients
64
65      F = fft(fz) / N;
66
67      %% Convert Fourier coefficients into derivatives
68
69      derivs = zeros(max_deriv + 1, 1);
70
71      for n = 0:max_deriv
72          derivs(n + 1) = factorial(n) / (r^n) * F(n + 1);
73      end
74  end
```

A.3 Fifth Derivative Finite Difference Function

Listing 3: MATLAB function for approximating the fifth derivative using a central finite difference stencil.

```
1   function fd5 = fifth_deriv_fd(f, x, h)
2   %FIFTH_DERIV_FD Approximate the fifth derivative using a central stencil.
3   %
4   %   fd5 = FIFTH_DERIV_FD(f, x, h) approximates the fifth derivative f^(5)(x)
5   %   using the central finite difference formula
6   %
7   %       [-f(x-3h) + 4f(x-2h) - 5f(x-h)
8   %        + 5f(x+h) - 4f(x+2h) + f(x+3h)] / (2*h^5).
9   %
10  %   Inputs:
11  %       f - function handle
12  %       x - point where the fifth derivative is approximated
13  %       h - step size, either a scalar or an array
14  %
15  %   Output:
16  %       fd5 - approximation to the fifth derivative
17  %
18  %   The implementation is vectorised with respect to h.
```

```
19
20      %% Input checks
21
22      if ~isa(f, 'function_handle')
23          error('Input f must be a function handle.');
24      end
25
26      if any(h <= 0)
27          error('All step sizes h must be positive.');
28      end
29
30      %% Fifth derivative central finite difference stencil
31
32      fd5 = ( ...
33          -f(x - 3.*h) ...
34          + 4.*f(x - 2.*h) ...
35          - 5.*f(x - h) ...
36          + 5.*f(x + h) ...
37          - 4.*f(x + 2.*h) ...
38          + f(x + 3.*h) ...
39          ) ./ (2.*h.^5);
40  end
```